UNIVERSITAS MUHAMMADIYAH SUKABUMI

Fakultas Sains dan Teknologi
Program Studi Teknik Informatika

MODUL PRAKTIKUM PERTEMUAN 4

Props dan State pada React Native

Props • useState • Data Flow • Event Handling • Derived State • Controlled Form

Mata Kuliah	Mobile Programming
Platform	React Native dengan Expo CLI
Studi Kasus	Diary App - lanjutan Praktikum Pertemuan 3
Fokus	Props, State, aliran data parent-child, dan UI dinamis
Pendekatan	Praktik bertahap, terstruktur, dan mudah direplikasi mahasiswa

Catatan utama: Praktikum ini melanjutkan project diary-app dari Pertemuan 3. Mahasiswa tidak membuat project baru. Seluruh perubahan dilakukan pada project yang sudah memiliki DiaryCard, DiaryDetailModal, design tokens, dan layout responsif.

A. Capaian Pembelajaran Praktikum

- Mahasiswa mampu menjelaskan perbedaan Props dan State pada React Native.
- Mahasiswa mampu mengirim data dari parent component ke child component melalui props.
- Mahasiswa mampu menggunakan useState untuk menyimpan data yang dapat berubah.
- Mahasiswa mampu menerapkan event handling dan callback props untuk mengubah state di parent.
- Mahasiswa mampu membedakan local state, shared state, dan derived state.
- Mahasiswa mampu membangun antarmuka Diary App yang lebih dinamis melalui fitur pencarian, like, pemilihan entri, dan penambahan data.

B. Prasyarat Praktikum

- Project diary-app dari Pertemuan 3 tersedia dan dapat dijalankan.
- Project menggunakan React Native berbasis Expo CLI.
- Struktur folder src/components, src/screens, dan src/styles tersedia.
- File DiaryCard.js, DiaryDetailModal.js, dan tokens.js sudah tersedia.
- Mahasiswa telah memahami View, Text, Image, Pressable, FlatList, StyleSheet, Flexbox, dan responsive layout.

Checkpoint: jalankan project dengan npx expo start. Pastikan Diary App Pertemuan 3 tampil tanpa error sebelum melakukan perubahan.

C. Ringkasan Studi Kasus

Pada Pertemuan 4, Diary App dikembangkan dari antarmuka responsif menjadi aplikasi yang memiliki data dinamis. Mahasiswa akan menggunakan props untuk mengirim data dan aksi dari parent ke child, sedangkan state digunakan untuk menyimpan data yang berubah selama aplikasi berjalan. Fitur yang dibangun meliputi pencarian entri, status suka (like), pemilihan entri untuk modal detail, serta penambahan entri baru melalui form terkontrol.

Target akhir: Diary App menampilkan data dari state, DiaryCard menerima data melalui props, pencarian memfilter daftar, tombol like mengubah state, modal detail menggunakan selected state, dan form dapat menambahkan entri baru tanpa me-restart aplikasi.

D. Output Praktikum

File/Output	Fungsi
src/components/DiaryCard.js	Presentational component yang menerima data dan callback melalui props.
src/components/SearchBar.js	Komponen pencarian dengan local state dan callback onChange.
src/components/EntryForm.js	Form terkontrol menggunakan state lokal.
src/screens/DiaryListScreen.js	Container component yang mengelola entries, query, likedIds, selected, dan derived state.
App.js	Integrasi screen utama pada project Expo tanpa Router.

E. Konsep Dasar Props dan State

Konsep	Penjelasan	Contoh pada Diary App
Props	Data/handler yang dikirim dari parent ke child. Child menerima props sebagai input dan tidak mengubahnya langsung.	title, date, liked, onToggleLike pada DiaryCard.
State	Data yang dimiliki komponen dan dapat berubah menggunakan setter dari useState.	entries, query, likedIds, selected.

Local State	State yang hanya dibutuhkan oleh satu komponen.	Isi TextInput pada SearchBar dan EntryForm.
Shared/Lifted State	State ditempatkan di parent agar dapat digunakan/dikendalikan beberapa child.	likedIds dan entries disimpan di DiaryListScreen.
Derived State	Nilai yang dihitung dari state lain, bukan disimpan sebagai state tambahan.	filtered dihitung dari entries dan query.
Callback Props	Fungsi dari parent yang dikirim ke child agar child dapat memicu perubahan di parent.	onChange, onSubmit, onToggleLike, onPress.

E.1. Mental Model Aliran Data

```
DiaryListScreen (PARENT / CONTAINER)
│
├── props data ──────> DiaryCard
│                      title, date, preview, liked
│
├── callback props <- DiaryCard
│                      onToggleLike(), onPress()
│
├── props callback -> SearchBar
│                      onChange()
│
└── props callback -> EntryForm
                       onSubmit()

Aksi pengguna
      ↓
callback dipanggil
      ↓
setState() di parent
      ↓
state berubah
      ↓
React melakukan re-render
      ↓
UI menampilkan data terbaru
```

Inti: Props mengalir dari parent ke child. Perubahan state dilakukan oleh komponen yang memiliki state tersebut. Child dapat meminta perubahan melalui callback props.

F. Langkah Awal Praktikum

F.1. Membuka Project Lanjutan

1. Buka Visual Studio Code.
2. Pilih File > Open Folder, lalu buka folder project diary-app dari Pertemuan 3.
3. Buka Terminal di VS Code dan pastikan terminal berada pada root project.
4. Jalankan project dengan perintah berikut.

```
npx expo start
```

Penting: jangan membuat project baru bernama prop-app. Praktikum ini sengaja melanjutkan diary-app agar mahasiswa melihat evolusi aplikasi dari pertemuan sebelumnya.

F.2. Struktur Folder yang Digunakan

```
diary-app/
├── App.js
├── assets/
└── src/
    ├── components/
    │   ├── DiaryCard.js
    │   ├── DiaryDetailModal.js
    │   ├── SearchBar.js
    │   └── EntryForm.js
    ├── screens/
    │   └── DiaryListScreen.js
    └── styles/
        └── tokens.js
```

G. Refactoring DiaryCard: Menerima Props

DiaryCard berperan sebagai presentational component. Komponen ini menerima data tampilan dan aksi dari parent melalui props. State bisnis seperti status like tetap dikelola oleh DiaryListScreen.

G.1. Edit src/components/DiaryCard.js

```
import React from 'react';
import { View, Text, Image, StyleSheet, Pressable } from 'react-native';
import { COLORS, SPACE, RADIUS, TYPO } from '../styles/tokens';

export default function DiaryCard({
  title,
  date,
  preview,
  moodImage,
  liked = false,
  onPress = () => {},
  onToggleLike = () => {},
```

```
}) {
  return (
    <Pressable
      onPress={onPress}
      style={({ pressed }) => [{ opacity: pressed ? 0.85 : 1 }]}
      accessible
      accessibilityLabel={`Entri ${title} tanggal ${date}`}
    >
      <View style={styles.card}>
        <Image source={moodImage} style={styles.mood} />

        <View style={styles.content}>
          <Text style={styles.title} numberOfLines={1}>{title}</Text>
          <Text style={styles.date}>{date}</Text>
          <Text style={styles.preview} numberOfLines={3}
ellipsizeMode="tail">
            {preview}
          </Text>
        </View>

        <Pressable
          onPress={onToggleLike}
          hitSlop={8}
          accessibilityLabel={liked ? 'Batal suka' : 'Suka'}
        >
          <Text style={styles.like}>{liked ? '❤️' : '🤍'}</Text>
        </Pressable>
      </View>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  card: {
    flexDirection: 'row',
    gap: SPACE.m,
    padding: SPACE.m,
    borderWidth: 1,
    borderColor: COLORS.border,
    borderRadius: RADIUS.card,
    backgroundColor: COLORS.surface,
    alignItems: 'center',
  },
```

```
  mood: {
    width: 64,
    height: 64,
    borderRadius: RADIUS.avatar,
    backgroundColor: '#eef2ff',
  },
  content: { flex: 1 },
  title: { fontSize: TYPO.title, fontWeight: '700', color: COLORS.text },
  date: { fontSize: TYPO.caption, color: COLORS.muted, marginBottom: 6 },
  preview: { fontSize: TYPO.body, color: COLORS.text },
  like: { fontSize: 22, paddingHorizontal: 6 },
});
```

Konsep: liked adalah props berupa data; onToggleLike adalah props berupa fungsi. DiaryCard tidak memiliki useState untuk status like.

G.2. Props yang Digunakan DiaryCard

Props	Jenis	Fungsi
title, date, preview	data	Menampilkan informasi diary.
moodImage	data	Sumber gambar lokal untuk mood.
liked	data boolean	Menentukan ikon hati terisi/kosong.
onPress	callback	Membuka modal detail.
onToggleLike	callback	Meminta parent mengubah status like.

H. SearchBar: Local State dan Callback Props

H.1. Buat src/components/SearchBar.js

```
import React, { useState } from 'react';
import { View, TextInput, StyleSheet } from 'react-native';
import { COLORS, SPACE, RADIUS } from '../styles/tokens';

export default function SearchBar({
  placeholder = 'Cari entri...',
  onChange,
}) {
  const [queryText, setQueryText] = useState('');

  const handleChange = (text) => {
    setQueryText(text);
    onChange?.(text);
  };

  return (
    <View style={styles.wrap}>
```

```
      <TextInput
        value={queryText}
        onChangeText={handleChange}
        placeholder={placeholder}
        style={styles.input}
        accessibilityLabel="Kolom pencarian"
      />
    </View>
  );
}

const styles = StyleSheet.create({
  wrap: {
    paddingHorizontal: SPACE.l,
    paddingVertical: SPACE.m,
  },
  input: {
    borderWidth: 1,
    borderColor: COLORS.border,
    borderRadius: RADIUS.card,
    paddingHorizontal: SPACE.m,
    paddingVertical: 10,
    backgroundColor: '#fff',
  },
});
```

queryText adalah local state milik SearchBar. Setiap perubahan input dikirim ke parent melalui callback onChange.

I. EntryForm: Controlled Form dengan State

EntryForm memiliki state lokal untuk setiap input. Saat tombol Simpan ditekan, data dikirim ke parent melalui onSubmit. Setelah berhasil dikirim, state form dikosongkan kembali.

I.1. Buat src/components/EntryForm.js

```
import React, { useState } from 'react';
import { View, TextInput, StyleSheet, Pressable, Text } from 'react-
native';
import { COLORS, SPACE, RADIUS, TYPO } from '../styles/tokens';

export default function EntryForm({ onSubmit }) {
  const [title, setTitle] = useState('');
  const [date, setDate] = useState('2026-10-19');
  const [preview, setPreview] = useState('');

  const canSubmit =
```

```
  title.trim().length > 0 &&
  preview.trim().length > 0;

const submit = () => {
  if (!canSubmit) return;

  onSubmit?.({
    id: Date.now(),
    title: title.trim(),
    date: date.trim(),
    preview: preview.trim(),
    body: preview.trim(),
    moodImage: require('../../assets/moods/default.png'),
  });

  setTitle('');
  setDate('2026-10-19');
  setPreview('');
};

return (
  <View style={styles.form}>
    <Text style={styles.heading}>Tambah Entri</Text>

    <TextInput
      value={title}
      onChangeText={setTitle}
      placeholder="Judul"
      style={styles.input}
    />

    <TextInput
      value={date}
      onChangeText={setDate}
      placeholder="YYYY-MM-DD"
      style={styles.input}
    />

    <TextInput
      value={preview}
      onChangeText={setPreview}
      placeholder="Ringkasan..."
      style={[styles.input, styles.multiline]}
```

```
        multiline
      />

      <Pressable
        onPress={submit}
        disabled={!canSubmit}
        style={[styles.button, !canSubmit && styles.buttonDisabled]}
      >
        <Text style={styles.buttonText}>Simpan</Text>
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  form: { padding: SPACE.l, gap: SPACE.s },
  heading: {
    fontSize: TYPO.title,
    fontWeight: '700',
    color: COLORS.text,
    marginBottom: SPACE.s,
  },
  input: {
    borderWidth: 1,
    borderColor: COLORS.border,
    borderRadius: RADIUS.card,
    paddingHorizontal: SPACE.m,
    paddingVertical: 10,
    backgroundColor: '#fff',
  },
  multiline: {
    minHeight: 80,
    textAlignVertical: 'top',
  },
  button: {
    marginTop: SPACE.s,
    padding: SPACE.m,
    borderRadius: RADIUS.card,
    backgroundColor: COLORS.accent,
    alignItems: 'center',
  },
  buttonDisabled: { opacity: 0.5 },
```

```
  buttonText: { color: '#fff', fontWeight: '700' },
});
```

Catatan aset lokal: siapkan file assets/moods/default.png. Penggunaan gambar lokal membuat praktikum tidak bergantung pada koneksi internet.

J. DiaryListScreen sebagai Container: Mengelola State Utama

DiaryListScreen menjadi sumber kebenaran (single source of truth) untuk data utama aplikasi. Komponen ini mengelola entries, query, likedIds, dan selected. Nilai filtered dihitung dari entries dan query sehingga tidak perlu disimpan sebagai state tambahan.

J.1. Edit src/screens/DiaryListScreen.js

```
import React, { useMemo, useState } from 'react';
import { View, Text, StyleSheet, FlatList } from 'react-native';

import { COLORS, SPACE, TYPO } from '../styles/tokens';
import DiaryCard from '../components/DiaryCard';
import DiaryDetailModal from '../components/DiaryDetailModal';
import SearchBar from '../components/SearchBar';
import EntryForm from '../components/EntryForm';

const INITIAL_ENTRIES = [
  {
    id: 1,
    title: 'Pagi yang Tenang',
    date: '2026-10-18',
    preview: 'Jalan pagi 20 menit dan menikmati udara segar.',
    body: 'Hari ini aku bangun lebih pagi dan berjalan kaki selama 20
menit.',
    moodImage: require('../../assets/moods/happy.png'),
  },
  {
    id: 2,
    title: 'Produktif di Kampus',
    date: '2026-10-17',
    preview: 'Menyelesaikan modul praktikum dan berdiskusi dengan tim.',
    body: 'Hari ini cukup produktif karena beberapa target perkuliahan
selesai.',
    moodImage: require('../../assets/moods/focus.png'),
  },
  {
    id: 3,
    title: 'Senja di Taman',
    date: '2026-10-16',
```

```
    preview: 'Membaca buku sambil menikmati suasana senja.',
    body: 'Sore ini saya menikmati waktu di taman sambil membaca buku.',
    moodImage: require('../../assets/moods/calm.png'),
  },
];

export default function DiaryListScreen() {
  // STATE UTAMA
  const [entries, setEntries] = useState(INITIAL_ENTRIES);
  const [query, setQuery] = useState('');
  const [likedIds, setLikedIds] = useState(new Set());
  const [selected, setSelected] = useState(null);

  // DERIVED STATE
  const filtered = useMemo(() => {
    if (!query.trim()) return entries;

    const q = query.toLowerCase();

    return entries.filter((entry) =>
      entry.title.toLowerCase().includes(q) ||
      entry.preview.toLowerCase().includes(q) ||
      entry.date.toLowerCase().includes(q)
    );
  }, [entries, query]);

  const toggleLike = (id) => {
    setLikedIds((prev) => {
      const next = new Set(prev);

      if (next.has(id)) {
        next.delete(id);
      } else {
        next.add(id);
      }

      return next;
    });
  };

  const addEntry = (newEntry) => {
    setEntries((prev) => [newEntry, ...prev]);
  };
```

```
  const renderItem = ({ item }) => (
    <DiaryCard
      title={item.title}
      date={item.date}
      preview={item.preview}
      moodImage={item.moodImage}
      liked={likedIds.has(item.id)}
      onToggleLike={() => toggleLike(item.id)}
      onPress={() => setSelected(item)}
    />
  );

  return (
    <View style={styles.container}>
      <Text style={styles.header}>Buku Harian</Text>

      <SearchBar onChange={setQuery} />

      <EntryForm onSubmit={addEntry} />

      <FlatList
        data={filtered}
        keyExtractor={(item) => String(item.id)}
        renderItem={renderItem}
        contentContainerStyle={styles.listContent}
        ListEmptyComponent={
          <Text style={styles.empty}>Belum ada entri.</Text>
        }
      />

      <DiaryDetailModal
        visible={!!selected}
        entry={selected}
        onClose={() => setSelected(null)}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
```

```
    backgroundColor: COLORS.surface,
  },
  header: {
    fontSize: TYPO.h1,
    fontWeight: '700',
    paddingHorizontal: SPACE.l,
    paddingTop: SPACE.l,
    color: COLORS.text,
  },
  listContent: {
    padding: SPACE.l,
    gap: SPACE.m,
    paddingTop: SPACE.m,
  },
  empty: {
    textAlign: 'center',
    color: COLORS.muted,
    marginTop: 40,
  },
});
```

J.2. Memahami State yang Dikelola

State/Nilai	Jenis	Peran
entries	State utama	Menyimpan seluruh data diary.
query	State utama	Menyimpan kata kunci pencarian dari SearchBar.
likedIds	State utama	Menyimpan ID entri yang disukai.
selected	State utama	Menyimpan entri yang sedang dibuka pada modal.
filtered	Derived state	Hasil filter dari entries dan query; dihitung dengan useMemo.

J.3. Mengapa Set Harus Dibuat Baru?

State likedIds menggunakan Set. Ketika status like berubah, jangan memodifikasi Set lama secara langsung. Buat Set baru agar React menerima referensi baru dan melakukan re-render dengan benar.

```
setLikedIds((prev) => {
  const next = new Set(prev);
  next.has(id) ? next.delete(id) : next.add(id);
  return next;
});
```

K. Integrasi ke App.js

Untuk project blank JavaScript tanpa Expo Router, gunakan App.js berikut.

```
import React from 'react';
import { StatusBar } from 'expo-status-bar';
import {
  SafeAreaProvider,
  SafeAreaView,
} from 'react-native-safe-area-context';

import DiaryListScreen from './src/screens/DiaryListScreen';

export default function App() {
  return (
    <SafeAreaProvider>
      <SafeAreaView style={{ flex: 1 }} edges={['top', 'left', 'right']}>
        <DiaryListScreen />
        <StatusBar style="dark" />
      </SafeAreaView>
    </SafeAreaProvider>
  );
}
```

L. Menjalankan dan Menguji Aplikasi

5. Pastikan file gambar lokal happy.png, focus.png, calm.png, dan default.png tersedia pada assets/moods/.
6. Pastikan semua file komponen sudah disimpan.
7. Buka terminal VS Code pada root diary-app.
8. Jalankan development server.

```
npx expo start
```

9. Uji pencarian dengan mengetik sebagian judul pada SearchBar.
10. Tekan ikon hati pada salah satu kartu dan pastikan status like berubah.
11. Tekan salah satu DiaryCard dan pastikan modal detail terbuka.
12. Isi EntryForm lalu tekan Simpan; pastikan entri baru muncul di bagian atas daftar.
13. Kosongkan kolom pencarian dan pastikan seluruh data kembali tampil.

Hasil yang diharapkan: perubahan props dan state langsung tercermin pada UI tanpa me-restart aplikasi.

M. Checklist Pengujian

- ☐ Project dapat dijalankan dengan npx expo start.
- ☐ DiaryCard menerima data melalui props.
- ☐ liked dan onToggleLike dikendalikan oleh parent.
- ☐ SearchBar memiliki local state dan mengirim perubahan melalui onChange.
- ☐ EntryForm menggunakan controlled inputs.
- ☐ EntryForm mengirim data baru melalui onSubmit.
- ☐ entries bertambah setelah form disimpan.
- ☐ query memfilter daftar tanpa mengubah data asli.
- ☐ likedIds berubah menggunakan Set baru.
- ☐ selected membuka dan menutup DiaryDetailModal.

- ☐ Tidak ada error import atau warning utama di terminal.

N. Troubleshooting Umum

Masalah	Kemungkinan Penyebab	Solusi Praktis
List tidak terfilter	query tidak diperbarui atau filtered tidak bergantung pada entries dan query.	Pastikan SearchBar memanggil onChange dan useMemo memiliki dependency [entries, query].
Like tidak berubah	Set lama dimutasi langsung.	Buat Set baru di dalam setter sebelum add/delete.
Form tidak reset	State input tidak dikosongkan setelah submit.	Panggil setter title/date/preview setelah onSubmit.
Entri baru tidak muncul	onSubmit tidak terhubung ke addEntry atau setEntries salah.	Pastikan <EntryForm onSubmit={addEntry} /> dan setEntries(prev => [newEntry, ...prev]).
Modal tidak muncul	selected tetap null atau onPress tidak memanggil setSelected(item).	Periksa props onPress pada DiaryCard.
Gambar lokal tidak tampil	Path require salah atau file belum tersedia.	Periksa assets/moods dan nama file secara persis.
Unable to resolve ../styles/tokens	Nama/path file token salah.	Pastikan file bernama tokens.js di src/styles/.

O. Tugas Praktikum

Kembangkan Diary App hasil Pertemuan 4 dengan ketentuan berikut:

- Implementasikan DiaryCard sebagai presentational component yang menerima data dan callback melalui props.
- Implementasikan SearchBar dengan local state dan callback onChange ke parent.
- Implementasikan EntryForm sebagai controlled form dan tambahkan entri ke state entries.
- Kelola entries, query, likedIds, dan selected pada DiaryListScreen.
- Tampilkan status like yang dapat berubah tanpa me-reload aplikasi.
- Gunakan gambar lokal pada assets/moods.
- Tambahkan minimal dua entri baru melalui form saat pengujian.
- Sertakan screenshot sebelum dan sesudah perubahan state.

P. Pengumpulan dan Rubrik Penilaian

P.1. Berkas yang Dikumpulkan

- Source code project tanpa folder node_modules jika dikompres.
- Screenshot fitur search, like, modal, dan penambahan entry.
- README singkat berisi nama, NIM, cara menjalankan project, dan penjelasan penggunaan props/state.

P.2. Rubrik Penilaian (100)

Aspek	Bobot	Indikator
Pemahaman Props	20	Data dan callback dikirim ke child dengan benar; child tidak memodifikasi props.
Pengelolaan State	30	useState digunakan tepat untuk entries, query, likedIds, selected, dan form.
Data Flow & Event Handling	20	Callback props, lifting state, dan update UI berjalan benar.
Kualitas Implementasi	20	Kode rapi, reusable, tidak terjadi mutasi state langsung, dan fitur berjalan.

Dokumentasi	10	README dan screenshot lengkap.

Q. Rangkuman

Pada Pertemuan 4, mahasiswa mempelajari bahwa props digunakan untuk mengirim data dan fungsi dari parent ke child, sedangkan state menyimpan data yang dapat berubah dan memicu re-render. DiaryCard digunakan sebagai presentational component, sementara DiaryListScreen bertindak sebagai container dan sumber kebenaran utama. SearchBar dan EntryForm memperlihatkan contoh local state, sedangkan callback props memperlihatkan bagaimana child dapat meminta parent memperbarui state. Dengan pola ini, Diary App berkembang dari antarmuka statis menjadi aplikasi yang responsif terhadap interaksi pengguna.

Pola utama yang perlu diingat: State dimiliki oleh komponen yang membutuhkan kontrol atas data; data dikirim ke child melalui props; aksi dari child dikirim kembali ke parent melalui callback props.

R. Referensi Teknis

- Modul Praktikum Pertemuan 4 - Props dan State (dokumen sumber yang diberikan).
- Modul Praktikum Pertemuan 3 - Diary App, desain UI dan layout responsif (sebagai prasyarat lanjutan).
- Dokumentasi React - Passing Props to a Component dan State: A Component's Memory.
- Dokumentasi React Native - TextInput, Pressable, FlatList, dan Image.